Examen Machine OS202

Santiago Florido Gomez
Ingénieur mécatronique
Étudiant STIC, ENSTA Paris

TABLE I
INFORMATIONS GÉNÉRALES DU PROCESSEUR

Champ	Valeur
Architecture	x86_64
Modes d'opération CPU	32-bit, 64-bit
Tailles d'adressage	46 bits physiques, 48 bits virtuels
Ordre des octets	Little Endian
Nombre de CPU logiques	16
Liste des CPU en ligne	0-15
Identifiant du fabricant	GenuineIntel
Nom du modèle	Intel(R) Core(TM) Ultra 7 155H
Famille CPU	6
Modèle	170
Threads par coeur	2
Coeurs par socket	8
Sockets	1
Stepping	4
BogoMIPS	5990.39
Virtualisation	VT-x
Fournisseur de l'hyperviseur	Microsoft
Type de virtualisation	full

TABLE II
CACHES ET TOPOLOGIE MÉMOIRE

Champ	Valeur
Cache L1d	384 KiB (8 instances)
Cache L1i	512 KiB (8 instances)
Cache L2	16 MiB (8 instances)
Cache L3	24 MiB (1 instance)
Nombre de noeuds NUMA	1
CPU du noeud NUMA 0	0-15

***Résumé*—Ce document, rédigé au format IEEE, présente une synthèse de l'environnement matériel observé depuis WSL pour les travaux du module OS202. Les informations ont été obtenues à l'aide de la commande `lscpu` et sont reformulées sous forme de tableaux en français.**

I. CARACTÉRISTIQUES DE L'ENVIRONNEMENT

Le Tableau I regroupe les informations générales sur l'architecture processeur visibles depuis WSL.

Le Tableau II présente les informations de cache et d'organisation NUMA disponibles dans la même sortie `lscpu`.

Ces valeurs ont été mesurées directement depuis WSL. Elles diffèrent donc d'une configuration à l'autre selon la machine hôte et les ressources allouées à l'environnement virtuel.

II. QUESTION PRÉLIMINAIRE

Il n'est pas représentatif, car la galaxie, dans une première approximation de simulation, peut être identifiée à une forme de disque plan qui ne présente pas une distribution considérable dans la direction Oz. C'est justement cette condition de concentration sur l'axe z qui fait que l'implémentation de davantage de couches en N_k conduise à une complication considérable de la grille et de la distribution des éléments dans la grille, sans que l'approximation faite du système ne soit améliorée de manière significative.

III. SECTION A PARALELIZER

Pour l'évaluation du temps que représente chacune des étapes de la simulation, il est proposé de mesurer les temps. À chaque itération, un temps est pris avant et après le rendu, ainsi qu'un autre avant et après la mise à jour de la simulation, en utilisant `time.perf_counter()`. Ces mesures sont prises à chaque frame, puis le temps consommé est moyenné toutes les 60 frames. Les résultats

```
[timing] 60 frames mesurées | avg_render=3.325 ms
[timing] 120 frames mesurées | avg_render=3.344 ms
```

montrent que la partie dominante du code se trouve dans le temps de mise à jour (*update*), qui inclut les calculs des accélérations ainsi que les mises à jour des positions et des vitesses. Étant donné qu'il s'agit de la section dominante de la simulation, on considère que la section de calcul et de mise à jour des corps à l'intérieur de la grille est la plus susceptible d'être parallélisée.

IV. SHARED MEMORY PARALELIZATION AVEC NUMBA

Étant donné que Numba est une méthode d'implémentation de la parallélisation en mémoire partagée, ce qui est proposé est de remplacer par `prange` les boucles dans lesquelles chacune des itérations correspond au résultat d'un corps distinct et qui écrivent dans des positions éloignées du tableau, afin d'éviter des phénomènes comme les blocages dus à la synchronisation, qui pourraient affecter la qualité des résultats de la parallélisation initiale. En suivant cette idée, il est proposé d'implémenter la parallélisation avec mémoire partagée (*Numba*) pour le calcul en plusieurs threads de l'accélération des corps ; pour cela, le décorateur est défini à `true` et, en outre, `range` est remplacé par `prange`. La parallélisation avec mémoire partagée n'est pas implémentée dans le calcul de la grille, car la présence de structures qui sont partagées, comme c'est le cas de `cell_counts`, `current_counts` et `body_indices`, pourrait donner lieu à des problèmes de

synchronisation dans la mémoire partagée et à une dégradation des résultats du calcul.

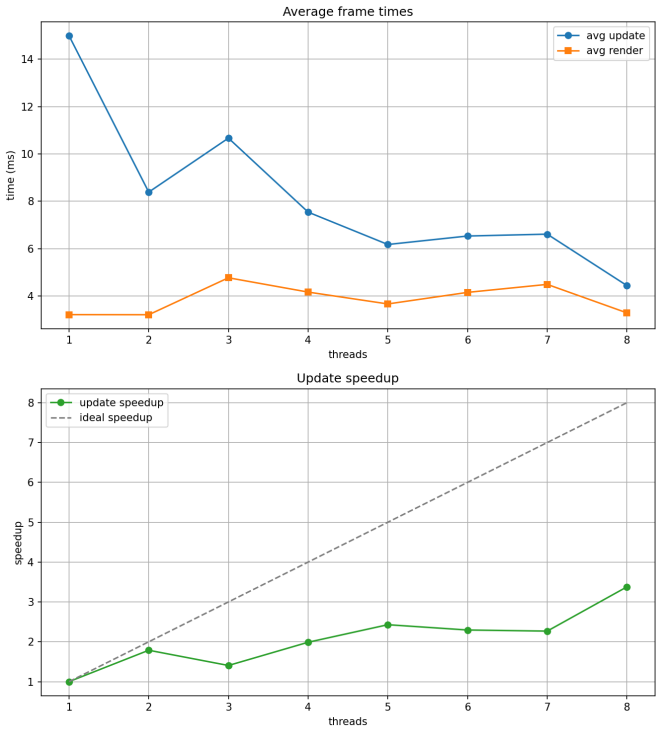

FIGURE 1. Temps moyens de rendu et de mise à jour, ainsi que l'accélération obtenue en fonction du nombre de threads Numba.

Les résultats montrent la diminution du temps moyen de rendu à mesure que l'on augmente le nombre de threads de calcul dans lesquels est répartie la boucle principale pour l'accélération de chaque corps. Néanmoins, on peut constater que le degré d'amélioration du temps de traitement à partir du deuxième thread s'éloigne du *speedup* idéal, ce qui peut être lié aux effets de synchronisation de la mémoire partagée, aux limitations imposées par les sections du code qui ne sont pas parallélisables, ou encore au fait que la dynamique de la grille n'a pas été modifiée en utilisant Numba, étant donné que la nature de ses accès mémoire ne la rendait pas idéale pour cette implémentation.

V. COMPUTE ET RENDER SEPARATION

Il est proposé une implémentation initiale de MPI en faisant usage de la parallélisation en mémoire partagée, de sorte qu'un nœud soit chargé du rendu et que le calcul s'exécute dans un second processus. Il convient de souligner que la parallélisation du calcul en sous-domaines n'est pas encore proposée ; on parle donc d'un unique processus additionnel.

Dans l'implémentation, exactement 2 rangs MPI sont conservés, l'un pour l'image et l'autre pour le calcul, et les mêmes paramètres de simulation de galaxie que dans le cas de Numba ont été employés. Il a été constaté, dans un premier temps, que MPI répartissait les CPU pour chaque processus MPI de manière équivalente, ce qui faisait que, dans le code, à partir d'un certain nombre de rangs avec Numba, des résultats différents de ceux obtenus avant la séparation apparaissaient, car MPI attribuait le même nombre de CPU au nœud de rendu, qui ne possède pas de parallélisation, et au nœud de calcul, qui, lui, en possède une, ce qui explique le *stalling* à partir de 4 threads. Une modification de l'affinité CPU est alors implémentée dans le contexte de l'exécution ; cette fonction fixe :

— le rang 0 au CPU 0,
— le rang 1 au reste des CPU 1..15,

afin que la répartition ne soit pas effectuée de manière automatique avec `mpiexec -bind-to none -n 2`. L'option `-bind-to none` évite qu'OpenMPI impose une liaison automatique qui gênerait l'exécution ; ensuite, le programme lui-même fixe l'affinité correcte avec `assign_rank_affinity`. On obtient alors le résultat attendu, dans lequel l'effet du temps de calcul et de traitement continue de dominer le temps de simulation et, en outre, s'ajoute le fait que, pour cette taille de galaxie, le temps de communication et de synchronisation entre les processus n'est pas très significatif, bien qu'il subsiste la nécessité imminente de pouvoir paralléliser les autres sections de l'algorithme de calcul qui ne sont pas adaptées à la parallélisation avec mémoire partagée, parmi lesquelles le traitement de la grille.

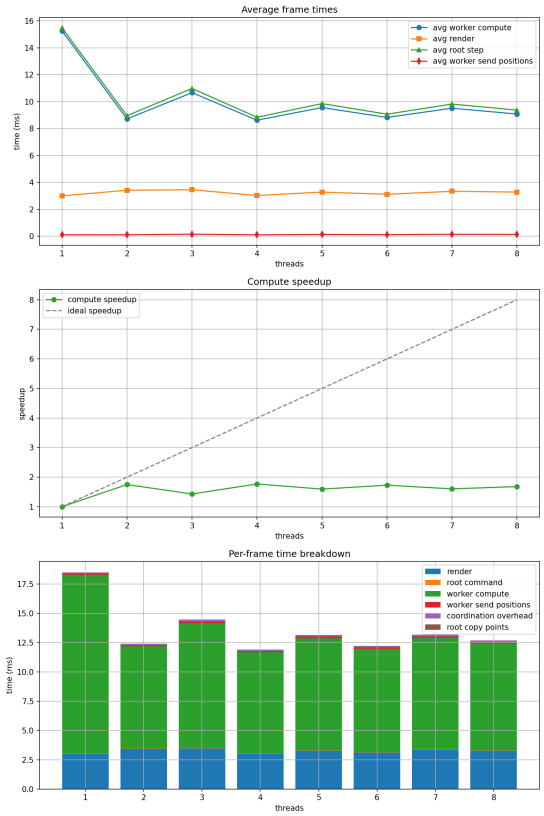

FIGURE 2. Mesures de la version MPI avec répartition symétrique des CPU entre les deux rangs.

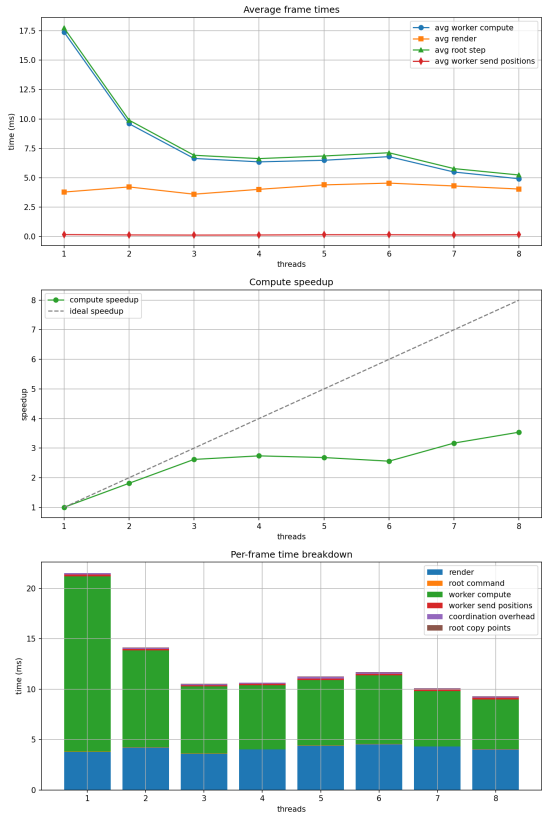

FIGURE 3. Mesures de la version MPI avec affinité CPU non symétrique, où le rang de calcul utilise les CPU restants.

VI. SUBDOMAIN DECOMPOSITION

A. Distribution de densité

Le fait que la distribution de la densité ne soit pas uniforme dans le domaine engendre un effet de déséquilibre dans la répartition des charges entre les différents sous-domaines qui sont implémentés. Cela a, à son tour, pour conséquence que certains processus prennent un temps de calcul plus élevé que les autres et, par conséquent, affectent la performance générale du système face à la parallélisation, compte tenu des conditions de charge auxquelles ils sont soumis. C'est pour cette raison qu'il est proposé, comme option alternative à une distribution dans un domaine cartésien 2D avec une répartition équitable et des sous-domaines de taille équivalente, d'implémenter une distribution en sous-domaines dont les dimensions ne soient pas nécessairement uniformes dans le domaine global, mais qu'elles s'adaptent plutôt aux particularités des conditions de charge, de manière à obtenir une distribution permettant d'avoir moins de cellules par processus dans la zone centrale du domaine global, puisque c'est là que la charge se concentre, et davantage de cellules par processus dans les autres zones. Le problème associé à cette implémentation est que la division des sous-domaines en sous-domaines incorporant un plus petit nombre de cellules par processus implique également un besoin plus important de communication dans la résolution des conditions de frontière entre les différents sous-domaines, ce qui s'accentuerait encore davantage dans la zone de plus forte densité, pouvant ainsi avoir un effet contre-productif sur les temps d'exécution.

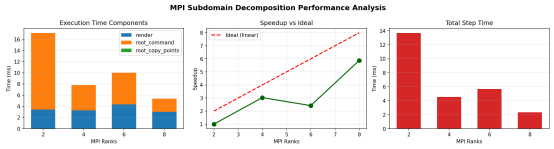

FIGURE 4. Mesures de performance de la version MPI par sous-domaines.

Les résultats de la figure 4 montrent une baisse nette du temps de calcul lorsque le nombre de rangs MPI augmente. Le temps moyen de commande du rang racine passe d'environ 13,68 ms pour 2 rangs à 4,52 ms pour 4 rangs, puis à 2,33 ms pour 8 rangs, ce qui correspond à un *speedup* apparent d'environ 3,03× puis 5,88× par rapport au premier cas mesuré. On observe toutefois un comportement non monotone à 6 rangs, où le temps remonte à 5,66 ms. Cela suggère qu'au-delà du gain apporté par la parallélisation, des effets de déséquilibre de charge, de coordination entre processus ou de répartition imparfaite du travail subsistent.

Il faut néanmoins interpréter ces mesures avec prudence, car la campagne enregistrée dans le fichier de résultats n'est pas strictement homogène : le cas à 2 rangs a été exécuté avec 2 threads, tandis que les cas à 4, 6 et 8 rangs ont été exécutés avec 16 threads. Malgré cette réserve, la tendance générale reste claire : la parallélisation réduit sensiblement le coût du calcul, mais le gain n'est pas parfaitement linéaire et le rendu, qui reste voisin de 3 à 4 ms, devient progressivement une composante non négligeable du temps total.

B. Conclusions

Les mesures montrent une amélioration nette du temps de calcul lorsque le nombre de rangs MPI augmente, avec le meilleur résultat observé à 8 rangs. Le temps moyen associé au calcul diminue fortement entre 2 et 4 rangs, puis atteint son minimum dans la configuration à 8 rangs. On peut donc conclure qu'il existe bien un effet positif de la parallélisation sur ce cas de test.

Cependant, le gain n'est ni parfaitement linéaire ni strictement monotone. Le cas à 6 rangs est moins performant que celui à 4 rangs, ce qui suggère la présence d'un déséquilibre de charge, d'un coût de coordination non négligeable entre processus, ou d'une répartition du travail qui n'est pas optimale pour cette taille de problème. La parallélisation améliore donc les performances, mais elle ne supprime pas les surcoûts liés à l'exécution distribuée.

Il faut également rester prudent dans l'interprétation quantitative de ces résultats, car la campagne de mesures enregistrée n'est pas complètement homogène du point de vue du nombre de threads utilisé dans chaque expérience. Il est donc plus rigoureux de parler d'une tendance favorable que d'une validation définitive d'un *speedup* idéal.

Enfin, le temps de rendu reste globalement compris entre 3 et 4 ms et devient une composante importante du temps

total à mesure que le calcul s'accélère. Pour une évaluation plus précise de l'effet propre de cette stratégie MPI, il serait souhaitable de relancer les benchmarks avec une configuration homogène sur tous les cas et de séparer encore plus clairement le coût du calcul de celui du rendu.

VII. Barnes–Hut

Dans ce cas, on peut proposer une distribution hiérarchique du quadtree entre les processus MPI. Comme le nombre de processus est une puissance de quatre, on peut choisir une profondeur L telle que $4^L = P$, où P est le nombre de processus. Alors, les boîtes de ce niveau sont réparties entre les processus, de sorte que chaque processus reçoive une sous-boîte ainsi que tout le sous-arbre qui en dépend. Les boîtes des niveaux plus élevés, comme la racine et les nœuds communs, peuvent être dupliquées dans tous les processus, ce que l'énoncé permet effectivement.

À chaque pas de temps, tous les processus calculent d'abord ensemble la boîte globale qui contient toutes les étoiles. Ensuite, chaque processus construit localement le sous-arbre de sa zone en utilisant les étoiles qui lui correspondent. Puis, une phase de communication avec MPI est réalisée afin d'échanger les informations importantes des nœuds de l'arbre, en particulier la masse totale et le centre de masse de chaque boîte. Ainsi, chaque processus peut disposer d'une vision globale du quadtree, du moins sous une forme résumée.

Après cela, le calcul des accélérations peut être parallélisé en répartissant les étoiles entre les processus selon cette même distribution spatiale. Chaque processus calcule l'accélération de ses propres étoiles. Lorsqu'une boîte lointaine satisfait le critère de Barnes–Hut, sa contribution est approximée en utilisant uniquement sa masse totale et son centre de masse. Si ce critère n'est pas satisfait, il faut alors descendre récursivement vers les sous-boîtes. Si l'on atteint une feuille contenant peu d'étoiles, on ne réalise plus d'approximation, mais on utilise directement les positions et les masses de ces étoiles pour effectuer le calcul.

Cette stratégie permet de paralléliser assez efficacement le calcul de l'accélération, mais elle entraîne également un coût supplémentaire de communication, puisqu'il est nécessaire de diffuser les informations de l'arbre entre les processus. De plus, un déséquilibre de charge peut apparaître si certaines sous-boîtes contiennent beaucoup plus d'étoiles que d'autres, par exemple celles qui sont les plus proches du centre de la galaxie.